Vysoké učení technické v Brně
Fakulta informačních technologií

Projektová dokumentace

Implementace překladače imperativního jazyka IFJ25

Tým x253038, varianta TRP-izp

SERHIJ ČEPIL	253038	25 %
DMYTRO KRAVCHENKO	XKRAVCD00	25 %
VERONIKA TURBAIEVSKA	XTURBAV00	25 %
MYKHAILO TARNAVSKYI	XTARNAM00	25 %

1 Rozdělení práce:

- Serhij Čepil
  - Lexikální analyzátor
  - Generátor vnitřního kódu (TAC)
  - Optimalizátor
- Dmytro Kravchenko
  - Lexikální analyzátor
  - Generátor cílového kódu
- Veronika Turbaievska (xturbav00)
  - Lexikální analyzátor
  - Syntaktický analyzátor
- Mykhailo Tarnavskyi (xtarnam00)
  - Lexikální analyzátor
  - Sémantický analyzátor

2 Struktura projektu:

- lexer.c
- main.c
- parses.c
- semantics.c
- tac.c
- optimizer.c
- codegen.c
- token.c
- symtable.c
- utils.c
- ast.c
- precedence.c
- stack_precedence.c
- expression.c
- ast.c
- printer.c

3 Předávání informací mezi částmi překladače

V projektu komunikují 5 hlavní častí (lexer, parser, semantics, tac a codegen). Začátek jde od lexeru, který dostává znaky z stdin. S znaků skládá lexémy. Lexémy převádí na jednotlivé tokeny s nějakým typem (identifikátor, int, klíčové slovo, plus, open paren atd.), hodnotou, pokud je potřeba, a pozicí v souboru. Lexer neodesílá všechny tokeny současně, odesílá tokeny zvlášť na vyžádání parseru.

Parser dostává jednotlivé tokeny a používá je k detekci gramatických konstrukce (výrazy, deklarace, příkazy atd.) a podle nich skládá abstraktní syntaktický strom. Uzly stromu reprezentují konstrukce (binární operace, cykly, podmínky atd.). Taky se generuje tabulka symbolů, kam se zapisuje data o identifikátorech (jméno, typ atd.). Když parser se skončí své práce předává tabulku symbolů a AST do sémantického analyzátoru.

Sémantického analyzátor dostává symbolů a AST a ověřuje sémantická pravidla: zda jsou proměnné deklarovány před použitím, zda typy operandů ve výrazech odpovídají atd. Do uzlů AST doplňuje zjištěné typy a další informace. V tabulce symbolů vytvoří úrovně vnoření identifikátorů a provede přejmenování podle úrovně. Pak úplně hotové AST a tabulku přijme generátor vnitřního kódu.

Generátor vnitřního kódu a optimalizátor přebírá AST a převádí složitější konstrukce na sekvenci jednoduchých instrukcí v formátě tříadresního kódu. Instrukce obsahuje operace a operadny (cíl a 2 argumentu), pokud operand je identifikátor přirazí odkaz na tabulku symbolu. Všechny instrukce budou uloženy do dvojsměrného seznamu. Použit dvojsměrného seznamu, protože někdy generátor cílového kódu potřebuje dívat do další instrukce a vrátit se zpět, např. při operace call pro ověření obsahu parametru funkce.

Generátor cílového kódu dostává tabulku symbolů a dvojsměrný seznam obsahující tříadresní kódy. Tabulku seznamů potřebuje pro deklarace na začátku globálné proměnné a pro generování typu framu proměnné. Podle operace z tříadresního kódu vypisuje na stdout sekvence příkazu v jazyce IFJcode25.

4 Vývojový cyklus, způsob práce v týmu

Pro většinu týmu to byl první týmový projekt související s programováním, proto bylo rozhodnuto, že nejprve všichni společně vytvoří lexikální analyzátor. Každý vytvořil svou část diagramu pomocí webu Lucidchart a implementoval ji. To všem pomohlo naučit se rychle komunikovat, používat git a vytvářet zprávy o práci.

Po lexikálním analyzátoru se konalo setkání týmu, na kterém se diskutovalo o formátu, v jakém bude každá část programu přijímat data. Nejprve byly vytvořeny struktury, které se používají mezi moduly. Když byly struktury vytvořeny, každý začal implementovat své moduly. Náš tým organizoval pravidelné offline schůzky, na kterých se nejprve diskutovali o progresu každého člena a poté všichni několik hodin programovali. Bylo rozhodnuto zorganizovat programovací schůze, protože to zajišťuje rychlou komunikaci a vzájemnou pomoc. Zvolili jsme právě offline setkání, protože to minimalizovalo technické problémy, zaručovalo maximální soustředění na projekt a teambuilding.

Před každým odevzdáním projektu byly organizovány schůze, během nichž se kontrolovala vzájemná interakce všech částí projektu a opravovaly chyby, které byly způsobeny nepřesností a subjektivností přirozeného jazyka.

5 Scanner (lexikální analyzátor)

Modul `lexer` je zodpovědný za převod vstupního zdrojového kódu na posloupnost tokenů, se kterými dále pracuje parser.

Hlavní funkcí souboru `lexer.c` je `scan_token`, která v závislosti na vstupním znaku a aktuálním stavu přepíná nebo zůstává v aktuálních stavech, dokud nedosáhne nepřípustného znaku ve stavu, pak lexer vrátí buď token, nebo chybu.

Problémy během vývoje

Pro sestavení tokenu klíčového slova jsme použili jednoduchý přístup. Klíčové slovo je podmnožinou identifikátorů, proto jsme na konec definice identifikátoru přidali kontrolu shody s klíčovými slovy

Nejvíce přepracovány byly analýza komentářů a řetězců. Diagram přechodu komentářů byl přepracován několikrát. V zásadě to souviselo s minimalizací počtu stavů a tokenů s hodnotou EOL, jakož i s odstraněním nedeterminismu z diagramu. Aby ve všech situacích následoval maximálně 1 EOL při každém vyhledávání tokenu, vynuluje se příznak nalezeného nového řádku. Tento příznak se používá, pokud po EOL začíná blokový komentář, pak na konci blokového komentáře se stav vrátí do EOL.

Pro uložení konstanty typu řetězec bylo nutné vytvořit funkci `write_str_trimmed`, která ukládá řetězec bez uvozovek. Algoritmus víceřádkového řetězce zahrnuje do přečtených znaků bílé znaky a zvyšuje počet znaků, které by neměly být zahrnuty na konci. V posledním týdnu jsme si všimli nedodržování výjimek pro víceřádkové řetězce a díky pohodlné struktuře souboru jsme to rychle opravili.

Komunikace s parserem

Parser komunikuje s funkcí `get_next_token`. Na začátku vývoje tato funkce pracovala pouze s `scan_token`. Během vývoje jsme zjistili, že parser někdy musí sledovat následující token a zároveň zpracovávat předchozí, proto nyní `get_next_token` pracuje s bufferem a volá `scan_token`, pokud je buffer prázdný.

6 Parser (syntaktický analyzátor)

Syntaxický analyzátor (parser) je zodpovědný za kontrolu struktury programu a vytvoření jeho mezivýsledného reprezentace **ve formě abstraktního syntaktického stromu (AST)**. Implementace je založena na hybridním přístupu, který kombinuje dvě metody analýzy:

- **Metoda rekurzivního sestupu (Recursive Descent):** Používá se pro zpracování hlavních konstrukcí jazyka (definice funkcí, cykly, podmíněné příkazy).
- **Precedenční analýza (Precedence Analysis):** Používá se výhradně pro zpracování výrazů (matematických, logických a řetězcových).

6.1 Struktura implementace

Zdrojový kód modulu je rozdělen do několika souborů podle logických úloh:

`parser.c` **(Hlavní modul):** Implementuje metodu rekurzivního sestupu na základě LL gramatiky a poskytuje jedinou veřejnou funkci `parser_run`, která spouští analýzu a vrací kořen AST programu.

`expression.c` **(Rozhraní pro výrazy):** Slouží jako rozhraní, které propojuje hlavní parser s modulem precedenční analýzy a předává mu zpracování výrazů a tvorbu jejich AST.

`precedence.c` **(Tabulka priorit):** Obsahuje implementaci tabulky priorit a definuje logiku přesunu/redukce pro korektní parsování matematických a logických výrazů.

`stack_precedence.c` **(Zásobník analýzy):** Implementuje specializovaný zásobník pro algoritmus shift-reduce, který uchovává operátory a uzly AST pro dynamickou konstrukci stromu výrazů.

6.2 Návrh syntaktického analyzátoru

- **Volba použití Abstract Syntax Tree (AST):** Abychom při generování kódu nemuseli složitě vracet zpět a doplňovat chybějící adresy, používáme AST. Tím se práce rozdělí do dvou kroků: nejprve vytvoříme strom a najdeme všechny funkce, a teprve potom generujeme kód. Tento postup dělá implementaci jednodušší a spolehlivější.

- **Důvod pro hybridní přístup:** Pro strukturální část jazyka (podmínky, cykly) jsme zvolili **rekurzivní sestup**, protože přirozeně kopíruje zanořenou strukturu kódu a je snadno laditelný. Naproti tomu pro výrazy je rekurzivní sestup neefektivní kvůli nutnosti řešit priority operátorů (což vede k hlubokému zanoření volání). Proto jsme pro výrazy zvolili **precedenční analýzu** (bottom-up), která priority zpracovává tabulkově a efektivněji.

7 Sémantický analyzátor

Sémantický analyzátor (modul `semantics.c`) provádí validaci logické správnosti programu nad vygenerovaným AST. Jeho úkolem je odhalit chyby, jako jsou nedefinované identifikátory, redefinice nebo typové neshody.

Analýza probíhá ve dvou fázích:

1. **Sběr deklarací:** Lineární průchod kořenovými uzly pro registraci hlaviček funkcí do globální tabulky symbolů.

2. **Analýza těl funkcí:** Hloubková kontrola příkazů a výrazů.

Tento modul prochází strom **rekurzivně**, přičemž využívá různé strategie průchodu v závislosti na typu uzlu:

- **Zpracování příkazů:** Funkce `analyze_statement` využívá průchod **pre-order**. Nejprve se zpracuje aktuální uzel (např. vytvoření nového rozsahu platnosti pro blok), a teprve poté se rekurzivně analyzují vnořené podstromy.

- **Vyhodnocení výrazů:** Funkce `analyze_expression` využívá průchod **post-order**. Nejprve se rekurzivně vyhodnotí typy operandů (listy stromu) a teprve na jejich základě se validuje operátor v nadřazeném uzlu.

Analyzátor kontroluje zejména:

- Definici všech použitých proměnných a funkcí.
- Unikátnost názvů v rámci jednoho rozsahu platnosti.
- Správný počet a typy argumentů při volání funkcí.
- Typovou kompatibilitu v aritmetických a relačních výrazech.

Použité datové struktury

Sémantický analyzátor využívá dvě hlavní datové struktury:

1. **Abstraktní syntaktický strom (AST)**
   Pro vnitřní reprezentaci programu byla zvolena struktura n-árního stromu implementovaná pomocí schématu *levý potomek, pravý sourozenec*. Každý uzel stromu (`AstNode`) obsahuje ukazatel na prvního potomka (`child`) a ukazatel na následujícího sourozence (`sibling`). Tato reprezentace byla zvolena proto, že umožňuje efektivně ukládat sekvence libovolné délky (např. seznamy parametrů funkcí nebo sekvence příkazů v bloku).

   Strom je budován syntaktickým analyzátorem s využitím pomocných konstruktorů a funkce `ast_node_add_child`. Tato funkce zajišťuje připojování nových uzlů na konec seznamu sourozenců, čímž je zachováno sekvenční pořadí příkazů tak, jak ve zdrojovém kódu.

2. **Zásobník rozsahů (ScopeStack)**
   Pro řízení viditelnosti identifikátorů a podporu vnořených bloků využívá sémantický analyzátor vlastní datovou strukturu `ScopeStack`. Jedná se o zásobník ukazatelů na jednotlivé instance tabulek symbolů (`Symtable*`), který dynamicky odráží aktuální kontext při průchodu stromem AST.

   Při rekurzivním sestupu do uzlu typu `NODE_BLOCK` (např. tělo podmínky `if`, cyklu `while` nebo funkce) je alokována nová lokální tabulka symbolů. Tato tabulka je následně vložena na vrchol zásobníku. Všechny nově definované proměnné v rámci tohoto bloku jsou ukládány do této vrcholové tabulky. Při opuštění bloku je tabulka ze zásobníku odebrána, čímž zaniká platnost lokálních proměnných.

   Klíčovým mechanismem je algoritmus vyhledávání identifikátorů (`H_Stack_Find_Var`). Pro nalezení definice proměnné se prohledává zásobník lineárně směrem od vrcholu (nejvnitřnější aktuální rozsah) ke dnu (vnější rozsahy).

8 Tabulka symbolů (symtable)

Tabulka symbolů slouží k ukládání a vyhledávání informací o identifikátorech (proměnných a funkcích) během překladu. Implementace se nachází v souborech `symtable.c` a `symtable.h`.

Implementace a řešení kolizí

Tabulka s rozptýlenými položkami je realizována jako dynamicky alokované pole struktur `TableEntry`. Pro mapování klíčů (názvů identifikátorů) na indexy v poli využíváme hašovací funkci **DJB2**, která je známá svou jednoduchostí a dobrou distribucí pro textové řetězce.

Jelikož se jedná o variantu s otevřenou adresací (open addressing), neukládáme kolizní prvky do spojových seznamů. Místo toho využíváme stavy jednotlivých slotů `SlotStatus`:

- `SLOT_EMPTY`: Slot je prázdný, hledání zde končí.
- `SLOT_OCCUPIED`: Slot obsahuje platná data.
- `SLOT_DELETED`: Slot obsahoval data, která byla smazána. Při vyhledávání tento stav přeskakujeme, ale při vkládání jej můžeme přepsat novým prvkem.

Abychom zachovali efektivitu vyhledávání $\mathcal{O}(1)$, tabulka sleduje svůj faktor naplnění. Pokud počet prvků přesáhne určitou mez kapacity, dojde k realokaci tabulky na větší velikost a k přehašování všech položek.

9 Generátor vnitřního kódu (TAC)

Modul `tac`(Three Address Code) realizuje transformaci abstraktního syntaktického stromu (AST) do tříadresového kódu (TAC). Tento mezikód slouží jako přechodná reprezentace mezi syntaktickou/semantickou analýzou a generováním cílového kódu (IFJcode25). Tříadresný kód používáme proto, že nám zjednoduší překlad do cílového kódu a umožní lepší optimalizaci. Generátor je implementován ve souborů `tac.c`.

Proces generování TAC

Vstupním bodem je funkce `generate_tac(...)`. Tato funkce přijímá kořen AST a rekurzivně prochází jeho uzly. Pro každý typ uzlu (např. přiřazení, podmínky, smyčky, volání funkcí) generuje odpovídající sadu TAC instrukcí.

Během průchodu stromem modul provádí následující klíčové operace:

1. **Linearizace výrazů:** Rozklad složených výrazů na elementární instrukce s dočasnými proměnnými(např. `$t1`, `$t2`).
2. **Řízení toku:** `if-else` a cyklus `while` jsou převedeny na lineární kód pomocí instrukcí skoků (`OP_JUMP`, `OP_JUMP_IF_FALSE`) a návěští (`OP_LABEL`). Funkce `create_unique_label` generuje unikátní názvy návěští (např. `L_ELSE_0`, `L_END_IF_1`).
3. **Funkce a volání:** Pro definice funkcí generuje speciální instrukce `OP_FUNCTION_BEGIN` a `OP_FUNCTION_END`.Parametry funkcí jsou zpracovány a předány jako argumenty instrukcí.

Použité datové struktury

Výstupem generátoru je dvousměrně vázaný seznam (`TACDLList`). Tato struktura reprezentuje seznam pro tzv. čtveřici (quadruple):

- `operation_code`: Typ operace.
- `result`, `arg1`, `arg2`: Operandy instrukce.

Operandy jsou reprezentovány strukturou `Operand`, která může nést různé typy dat definované výčtem `OperandType`. Díky přímému propojení operandů typu `SYMBOL` se záznamy v tabulce symbolů (`TableEntry`) nemusí generátor cílového kódu provádět opakované vyhledávání názvů, což zvyšuje efektivitu překladu.

10 Optimalizace mezikódu (optimizer)

Modul `optimizer` má za úkol vylepšit vygenerovaný tříadresný kód (TAC) ještě předtím, než se převede do finálního IFJcode25. Cílem je zmenšit počet instrukcí a zrychlit běh programu, aniž bychom změnili jeho logiku. Implementace se nachází v souborech `optimizer.c` a `optimizer.h`.

Optimalizace probíhají v cyklu nad seznamem instrukcí. Celý proces se opakuje tak dlouho, dokud v kódu dochází k nějakým změnám.

Použité optimalizační techniky

Náš optimalizátor využívá kombinaci čtyř základních metod:

- **Constant Propagation (Šíření konstant):** Pokud překladač zjistí, že proměnná má v daném místě pevně danou hodnotu (např. `a = 5`), nahradí všechna její následná použití přímo touto hodnotou. K tomu využíváme pomocnou mapu hodnot.
- **Constant Folding (Skládání konstant):** Pokud instrukce provádí operaci nad dvěma konstantami (např. `3 + 2` nebo spojení řetězců `"A" + "B"`), optimalizátor výsledek spočítá rovnou při překladu a nahradí instrukci přiřazením výsledku.
- **Unreachable Code Elimination (Odstranění nedosažitelného kódu):** Odstraňujeme instrukce, které se nikdy neprovedou. To nastává typicky za instrukcemi nepodmíněného skoku (`JUMP`) nebo návratu z funkce (`RETURN`), pokud za nimi bezprostředně nenásleduje návěští.
- **Dead Code Elimination (Odstranění mrtvého kódu):** Procházíme kód a počítáme, kolikrát je každá proměnná použita. Pokud zjistíme, že se do proměnné zapisuje, ale nikde se nečte, a není to např. volání funkce nebo výpis, instrukci odstraníme.

Pro efektivní fungování těchto metod využívá modul pomocné struktury, jako je `PropagatorMap` pro sledování hodnot proměnných a `VarUsage` pro počítání referencí při eliminaci mrtvého kódu.

11 Generátor cílového kódu (gencode)

Modul `codegen` realizuje sestavení cílového kódu ve formátu IFJcode25 z instrukcí tříadresového kódu. Na rozdíl od implementace, kde je cílový kód generován přímo z AST, jsme se rozhodli generovat také vnitřní kód. Mezikód pomohl oddělit generátor od syntaktické a sémantické analýzy, takže bylo pohodlnější soustředit se na generování kontrol běžných chyb (chyb typů) a testování.

Generátor je implementován ve dvou souborech: 1. `codegen.h` obsahuje deklaraci veřejných funkcí (`generate_code`, `string_to_ifjcode`), 2. `codegen.c` obsahuje implementace veřejných a lokálních funkcí.

Generování kódu

Funkce `generate_code(...)` je vstupem do generátoru cílového kódu. Prochází seznamem instrukcí a v závislosti na operaci volá potřebnou funkci, přičemž každá operace z TAC má svou vlastní funkci. Každá funkce generuje potřebné kontroly typů a převody int na float nebo

float na int, pokud float v desetinné části obsahuje nulu. Tento přístup pomáhá při testování a zvyšuje čitelnost kódu.

Každá funkce operace používá menší funkce pro generování jednotlivých instrukcí, například `gen_defvar`, `gen_move`. Může také volat funkce, které generují hotovou sérii instrukcí (`gen_type`, `gen_same_operand_check` atd.) nebo funkce pro generování části instrukce (`gen_operand`).

Generátor také vytváří své jedinečné návěští pomocí externí funkce `create_unique_label`, která pokračuje v číslování po generátoru vnitřního kódu.

Použité datové struktury

Modul používá dvě hlavní datové větve. První větev tvoří instrukce. Jsou implementovány pomocí TacInstruction, který obsahuje datové typy TacOperationCode a Operand. Tyto dvě struktury jsou pro generování klíčové, protože obsahují názvy návěští, identifikátory, konstanty, operace atd. Každá instrukce obsahuje odkaz na seznam TACDLList.

Druhou větví je tabulka symbolů. Pomáhá na začátku generování generovat globální proměnné, protože jsou uloženy na nejvyšší úrovni tabulky, a určovat rámec proměnné. Pokud proměnná není v nejvyšší úrovni, musí být uložena v lokálním rámci, jinak v globálním. Tabulka symbolů také obsahuje jedinečná jména pro každou proměnnou a funkci.

12 LL Gramatika

1. `<Program> -> <prolog> <kostra> EOF`
2. `<prolog> -> import "ifj25" for Ifj EOL`
3. `<kostra> -> class Program { EOL <func_list> }`
4. `<func_list> -> <function> <func_list>`
5. `<func_list> -> epsilon`
6. `<function> -> <func_name> <func_tail>`
7. `<func_name> -> static id`
8. `<block> -> { EOL <func_body> }`
9. `<func_tail> -> ( <params> ) <block>`
10. `<func_tail> -> = ( <params> ) <block>`
11. `<func_tail> -> <block>`
12. `<params> -> <id> <param_tail>`
13. `<params> -> epsilon`
14. `<param_tail> -> , id <param_tail>`
15. `<param_tail> -> epsilon`
16. `<func_body> -> <statement> <func_body>`

17. <func_body> -> epsilon
18. <statement> -> return <expression> EOL
19. <statement> -> if (<expression>) <block> else <block>
20. <statement> -> var id EOL
21. <statement> -> while (<expression>) <block>
22. <statement> -> <block>
23. <statement> -> global_id EOL
24. <statement> -> id = <right_side>
25. <statement> -> global_id = <right_side>
26. <statement> -> epsilon
27. <right_side> -> id (<list_terms>)
28. <right_side> -> global_id (<list_terms>)
29. <right_side> -> ifl . <builtin> (<list_terms>)
30. <right_side> -> <expression>
31. <list_terms> -> <term> <terms_tail>
32. <list_terms> -> epsilon
33. <terms_tail> -> , <term> <terms_tail>
34. <terms_tail> -> epsilon
35. <builtin> -> read_str
36. <builtin> -> read_num
37. <builtin> -> write
38. <builtin> -> floor
39. <builtin> -> str
40. <builtin> -> length
41. <builtin> -> substring
42. <builtin> -> strcmp
43. <builtin> -> ord
44. <builtin> -> chr
45. <term> -> id

46. `<term> -> literal_num`

47. `<term> -> literal_string`

48. `<term> -> null`

13 LL Rozborová tabulka

Non-Term	imp	cls	statc	}	{	(	)	=	,	ret	if	else	var	while	id	gid	lit	null	ifl	blt	EOL	EOF
<Program>	1																					
<prolog>	2																					
<kostra>		3																				
<func_list>			4	5																		
<function>			6																			
<func_name>			7																			
<func_tail>					11	9		10														
<block>					8																	
<params>							13								12							
<param_tail>							15		14													
<func_body>				17	16					16	16		16	16	16	16						
<statement>					22					18	19		20	21	24	**23,25**						
<right_side>															**27,30**	28	30	30	29			
<list_terms>							32								31		31	31				
<terms_tail>							34		33													
<term>															45		46,47	48				
<builtin>																				35-44		

Legenda: imp: import, **cls**: class, **statc**: static, **ret**: return, **gid**: global_id, **lit**: literal_num/string, **blt**: názvy vestavěných funkcí (read_str atd.).

14 Precedenční tabulka (Precedence Table)

	+	-	*	/	<	>	<=	>=	is	==	!=	(	)	i	k	$
+	>	>	<	<	>	>	>	>	>	>	>	<	>	<		>
-	>	>	<	<	>	>	>	>	>	>	>	<	>	<		>
*	>	>	>	>	>	>	>	>	>	>	>	<	>	<		>
/	>	>	>	>	>	>	>	>	>	>	>	<	>	<		>
<	<	<	<	<					>	>	>	<	>	<		>
>	<	<	<	<					>	>	>	<	>	<		>
<=	<	<	<	<					>	>	>	<	>	<		>
>=	<	<	<	<					>	>	>	<	>	<		>
is	<	<	<	<	<	<	<	<		>	>		>		<	>
==	<	<	<	<	<	<	<	<	<			<	>	<		>
!=	<	<	<	<	<	<	<	<	<			<	>	<		>
(	<	<	<	<	<	<	<	<	<	<	<	<	=	<		
)	>	>	>	>	>	>	>	>	>	>	>		>			>
i	>	>	>	>	>	>	>	>	>	>	>		>			>
k	>	>	>	>	>	>	>	>	>	>	>		>			>
$	<	<	<	<	<	<	<	<	<	<	<	<		<		=

Legenda: i: identifikátor/literál (Num/String), **k**: klíčová slova typů/null (index 14), **$**: konec vstupu/zásobníku. Prázdné buňky značí chybu (E).

15 Tabulky a diagram
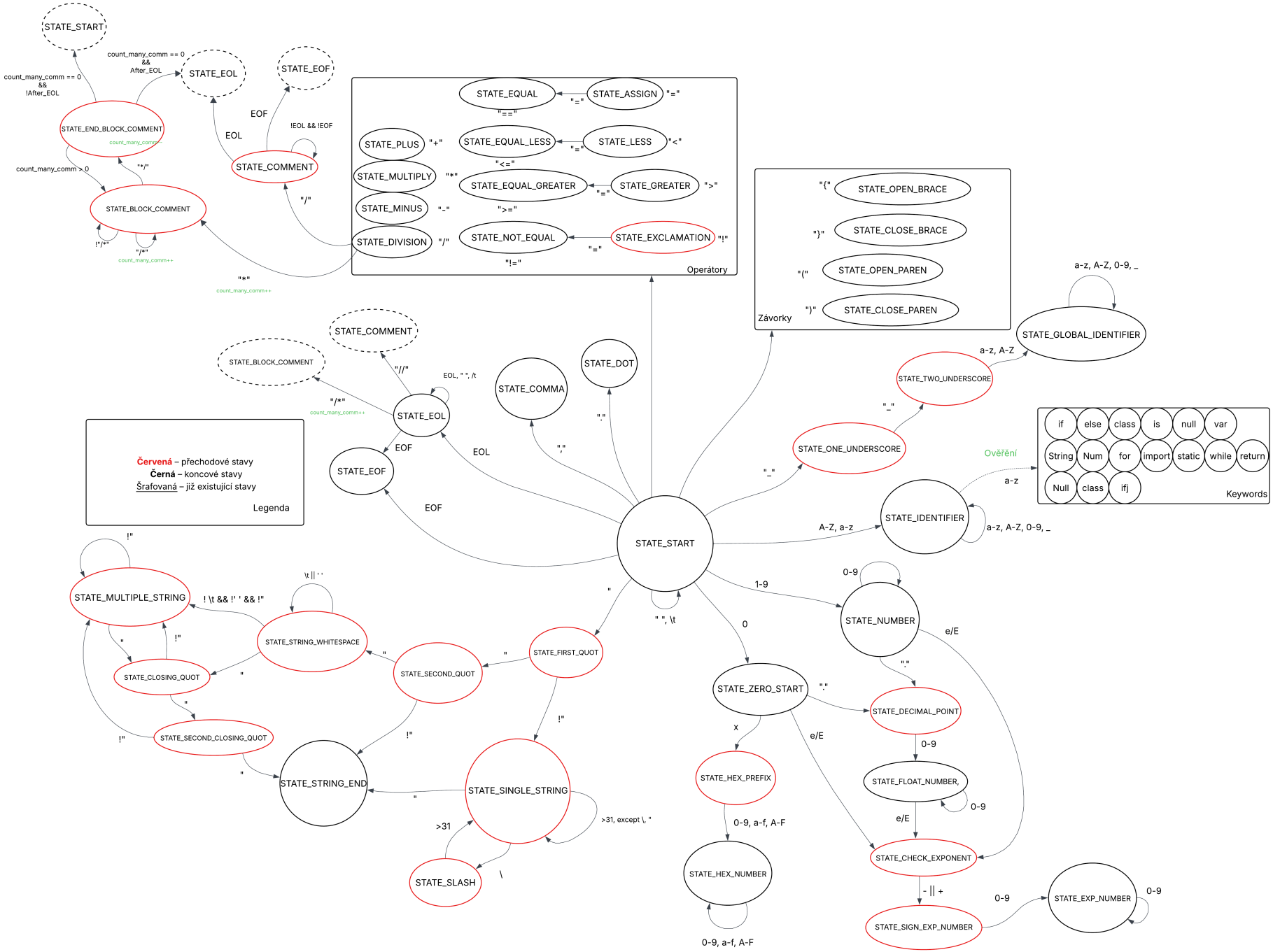
Diagram konečného automatu, který specifikuje lexikální analyzátor.